



第19章： 智能体安全与伦理展望



学习目标与主线

学习目标与主线

01

识别主线：理解 Agent 相比 Chatbot 的新增风险面——“能行动 + 有记忆 + 吃外部输入”，因此攻击会从注入扩展到越权、持久化与外泄

02

掌握方法：掌握可落地的纵深防御栈——最小权限与分级授权、注入隔离、记忆治理、预算/熔断、沙箱与审计

03

形成闭环：建立治理与合规的安全闭环——自主权边界 + 人机审批设计 + 红队评测与在线监控 + 事故响应，支撑行业上线



总览

为什么 Agent 风险不同于纯聊天模型

重点

Chatbot 的风险主要是“输出错误信息”，Agent 的风险是“把错误变成行动、状态、传播链条”

- Chatbot: 输出文本为主，最多“建议你做什么”
- Agent: 能调用工具（发邮件/下单/改仓库/改配置/转账/创建工单），输出会直接触发真实世界变化
- 风险升级点：错误或被注入的指令不再停留在“说”，而是变成“做”（不可逆、可连锁）

1) 行动能力 (Actionability)

- Chatbot: 相对“短会话”，上下文多为临时；即便说错，通常不会自动在未来再次触发
- Agent: 有记忆、任务队列、计划（甚至跨天跨周持续运行），错误可能被写入状态后**反复放大**
- 风险升级点：从一次性失误变成“持久化风险”（今天埋、下周炸；一次污染，多次触发）

2) 长期状态 (Statefulness)

- Chatbot: 主要输入来自用户对话
- Agent: 会摄取网页/文档/邮件/工单/日志/传感器等外部内容，并可能把内容回灌进上下文当“依据”
- 风险升级点：出现间接注入与供应链式攻击：攻击者不必直接对话，只要污染网页/邮件/文档就能影响行为

3) 外部输入面 (External Attack Surface)

- 因为 Agent 有“做事能力”，所以必须有**权限/预算/确认/审计**（否则就是“无限授权机器人”）
- 因为 Agent 有“长期状态”，所以必须有**记忆写入策略**（字段、TTL、来源标注、审批）
- 因为 Agent 有“外部输入面”，所以必须有**内容/指令分离与检索隔离/清洗**（外部内容默认不可信）

威胁模型与攻击面总览



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

资产 (Assets)

- **凭证/密钥**——API key、OAuth token、云账号临时凭证、数据库密码、SSH key
 - 风险：一旦被拿走，攻击者不需要再操控模型，直接“拿钥匙进门”
- **私有数据**——用户隐私、企业文档、邮件、工单、客户信息、代码与内部知识库
 - 风险：合规与商业损失（数据外泄、越权访问、跨租户泄露）
- **钱包/预算**——付费 API、云资源费用、检索/浏览/代码执行成本、转账/下单权限
 - 风险：Denial-of-Wallet（把钱烧光）或引导资金流向错误目的地
- **生产系统权限**——CI/CD、仓库写权限、K8s/云控制台、运维系统、工单系统、支付/财务系统
 - 风险：最严重（篡改配置、植入后门、删库、停机事故）



Attack Surface

每层“攻击者能做什么”

- **Prompt 层**（指令入口）
 - 攻击：直接注入、角色混淆、越权指令、提示绕过
- **RAG / 记忆层**（证据与状态层）
 - 攻击：间接注入（网页/文档藏指令）、记忆投毒embedding/向量检索弱点、跨租户召回
- **Tool 层**（行动接口层）
 - 攻击：诱导调用高权限工具、参数注入（危险参数/目的地）、输出处理不当导致执行
- **环境层**（Runtime/系统边界）
 - 攻击：沙箱逃逸、滥用网络 egress、文件系统/日志侧漏、依赖供应链污染
- **人与流程层**（People & Process）
 - 攻击：社工、审批疲劳、权限配置错误、值班响应缺失、审计不可回放

关键理解——“资产 × 攻击面”

➤ **攻击者目标**：拿到 Secrets / Data / Money / Privileges（资产）

➤ **攻击路径**：从 Prompt 或外部内容进入 → 污染 Retrieval/Memory → 触发 Tool 执行 → 借 Environment 扩大 → 利用流程漏洞持续化



威胁清单与典型攻击链

权威风险清单：OWASP LLM Top 10 (2025) 如何映射到 Agent



一句话定位

OWASP 2025 这 6 类风险，基本覆盖了 Agent 最常见的“注入→越权→执行→泄露/烧钱”的主路径：Prompt / RAG(向量&记忆) / Tool 执行 / 自主权 / 成本边界

OWASP 风险（你选的6类）	Agent 典型入口/触发点	资产/后果（更像事故描述）	首要控制点（工程落地）	评测/监控信号（上线前/后）
LLM01 Prompt Injection	用户对话 / 网页 / 邮件 / 文档 (间接注入)	越权调用工具、规则绕过、行动被劫持	指令/数据分离 系统提示不可被外部覆盖 高风险动作二次确认	注入成功率；“忽略规则/越权意图” 触发次数；高危tool-call占比
LLM02 Sensitive Info Disclosure	RAG召回、工具返回、日志/记忆回灌	泄露密钥/PII/内部文档 (跨租户更严重)	输出脱敏与红线检测 检索权限绑定 (user/tenant) 最小化上下文	敏感串命中率；脱敏触发次数；跨租户召回拦截数
LLM05 Improper Output Handling	LLM 输出被当成 可执行输入 (SQL/shell/URL/JSON)	命令注入、危险参数执行、错误工单/配置落库	Tool Gateway 参数校验+allowlist 结构化输出schema校验 转义/编码	“校验失败/拦截”比例；危险参数命中；执行前后diff异常
LLM06 Excessive Agency	权限过大、自动化过强 (默认写/默认执行)	不可逆操作 (改仓库/改配置/转账/删除资源)	最小权限+能力分级 (read/write/money) 审批聚合 (证据包) kill switch	高权限动作频率；人工批准率/拒绝率；降级触发次数
LLM08 Vector & Embedding Weaknesses	向量库投毒、相似度操控、召回越权	被操控输出 长期“错误事实/偏好” 数据混租泄露	记忆写入策略 (字段/TTL/来源) 检索清洗与引用隔离 向量库ACL	“可疑来源”召回占比；投毒样本命中；记忆写入拒绝率
LLM10 Unbounded Consumption	循环计划、反复检索/重试、超长输入	DoS/烧钱/服务退化 (Denial-of-Wallet)	Budget (token/时间/钱/工具次数) Loop limits 熔断+退避+降级	单任务cost分布；tool-call回路长度；异常速率告警/熔断次数

Agent 攻击链：从注入到持久化与外泄

1) 初始进入：直接/间接注入 (Entry)

- 动作：把恶意指令塞进“你会读的输入”。
 - 直接：用户对话里下黑指令
 - 间接：网页/邮件/文档里藏指令，RAG/浏览器把它当内容回灌
- 目标：劫持意图/规则，让模型把“外部内容”当“高优先级指令”
- 结果：从“说什么”开始偏航

2) 越权执行：诱导调用高权限工具

- 动作：诱导 Agent 触发高权限工具（写库/发邮件/改配置/拉数据/转账）
- 目标：把文本层面的操控升级成“真实行动”
- 结果：哪怕只成功一次，也可能产生不可逆后果（改了配置、发出了邮件、创建了凭证）

3) 持久化：写入长期记忆/知识库

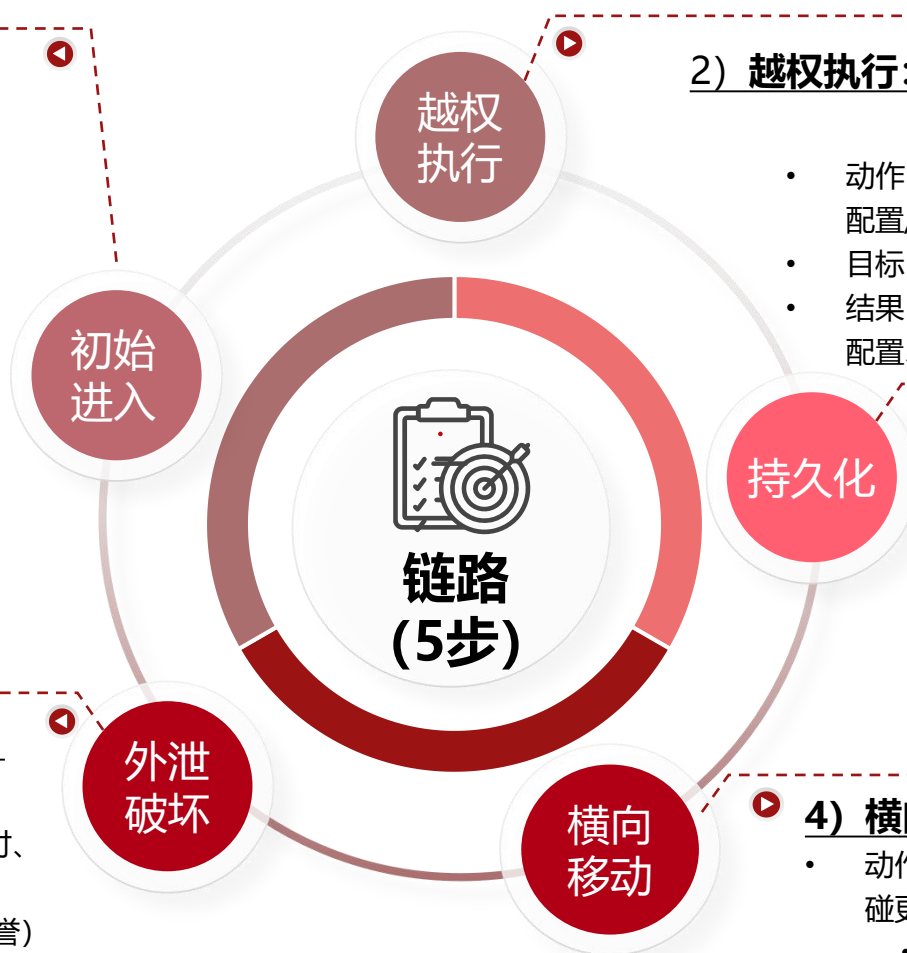
- 动作：让 Agent 把恶意内容写进长期状态（Memory/KB/任务队列/偏好设置）
- 目标：跨会话、跨时间持续触发（今天埋，下周自动炸）
- 结果：攻击从一次性变成“自我复制的后门”

4) 横向移动：借工具/凭证访问更多系统

- 动作：利用已经拿到的工具能力/临时凭证/导出的信息，去碰更大权限域：
 - 从“工单系统”到“邮件/知识库”，再到“云控制台/CI/CD”
- 目标：扩大资产覆盖面（更多数据、更多密钥、更高权限）
- 结果：影响面从单点变成多系统连锁

5) 外泄/破坏：数据泄漏、资金损失、生产事故

- 动作：导出敏感数据、发送到外部、触发支付、删资源、植入后门
- 目标：最终变现或破坏（钱/数据/可用性/声誉）
- 结果：真正的“安全事故”落地（合规、财务、生产都可能中招）





工具调用安全与可落地控制面

工具调用的 AuthN/AuthZ: 最小权限与能力隔离

AuthN vs AuthZ

1

AuthN (Authentication) = 你是谁

工具网关要知道: 是谁在发起调用 (用户/租户/Agent实例/任务ID), 以及这次调用来自哪个会话/哪条指令链

2

AuthZ (Authorization) = 你能做什么

不仅是“能不能调用某工具”, 更是“能调用哪些方法、哪些参数范围、哪些资源” (细粒度 scope)

3

核心:

Agent 的权限不是“给一个 token 就完了”, 而是每次 tool-call 都要带着可验证身份、可审计上下文和可约束的权限范围

最小权限 (Least Privilege)

1

工具权限按能力拆分

- 目的: 把“低风险能力”和“高风险能力”物理隔离, 避免一个 token 通吃
- read: 查工单、读文档、检索——write: 改配置、写DB、提交PR——money: 支付、下单、开通付费资源——irreversible: 删除资源、发布到生产、发外部邮件群发

2

细粒度 scope + 短期 token

- scope 要细到: 资源范围——操作范围——参数范围
- 短期 token:
 - 绑定任务/会话/用户/租户, 并设置 TTL (分钟级)
 - 用完即失效, 降低“拿到一次凭证就长期横向移动”的风险

3

高风险动作“分级授权”

- 红黄绿分级 (也可叫“自动化等级”)
- 避免批准疲劳的小设计: 风险聚合审批+预览和风险确认

总结

Agent 的安全控制面核心是“把工具当成 API 安全问题”: 身份要可验证 (AuthN), 权限要可约束 (AuthZ), 并用**最小权限** + **分级授权**把“能做什么、做到哪一步”锁死

Prompt Injection（直接/间接）：核心难点与原则

1) 直接注入 (Direct Injection)

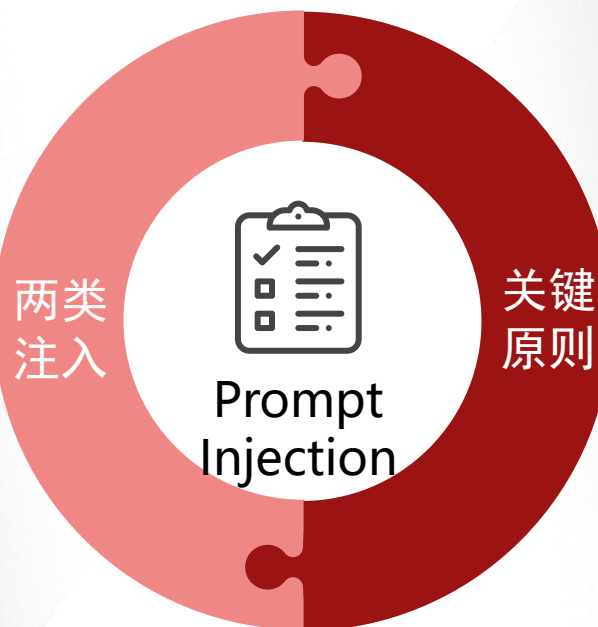
- ✓ 发生位置：用户对话框里。
- ✓ 典型手法：要求忽略规则、伪装系统消息、诱导泄露、诱导执行高风险动作。
- ✓ 为什么能成：模型在语言层面“很会配合”，且用户输入天然看起来像指令。

2) 间接注入 (Indirect Injection) ——Agent 的核心难点

- ✓ 发生位置：网页/文档/邮件/工单/代码注释等外部内容。
- ✓ 危险点：Agent 会“读外部内容 + 检索回灌”，外部内容被混进上下文后，模型分不清
- ✓ 最常见的致命误区：RAG 把外部内容当成可信知识，从而把“攻击者的指令”变成“系统的一部分”。

直接注入

间接注入



外部内容默认不可信：只作为数据/证据参与推理，绝不作为指令改变行为

- ✓ 把 prompt **结构化**成三块，显式告诉模型：检索内容里出现“命令式文本”也只能当引用，不得执行
- ✓ 检索内容的“**引用化**”呈现：强制加来源标签+强制用“quote block”语义
- ✓ 工具调用必须过“工具网关”：即使模型被注入，也要在 Tool 层用 allowlist/参数校验/权限分级拦住

工具调用——过网关

检索内容的“引用化”

上下文分区

总结

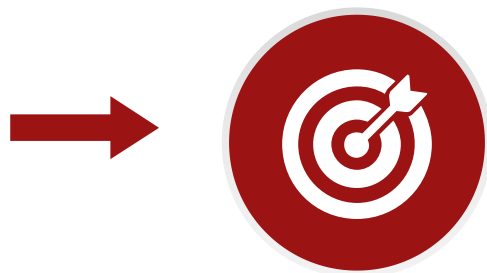
Prompt Injection 的**本质**是“指令优先级被劫持”
防护的**第一原则**是：外部内容一律当数据，不当指令——必须做**内容/指令分离**

记忆与RAG特有风险

“记忆” Agent 高危点

为什么“记忆”是 Agent 特有高危点？

- 可跨时间触发（今天埋、下周炸）
 - 1) 与 Prompt 注入不同：Prompt 注入通常依赖“当前上下文”
 - 2) 记忆投毒是把攻击写进“未来会被自动检索的状态”，让攻击变成默认行为的一部分
- 更像“自己学到的事实/偏好”（高信任来源错觉）
 - 1) 模型/系统往往会把记忆当作“用户偏好/历史事实”，优先级甚至高于即时对话
 - 2) 这会导致：攻击者把指令伪装成“偏好/规则/常用联系人/常用地址”，系统后续很难察觉异常

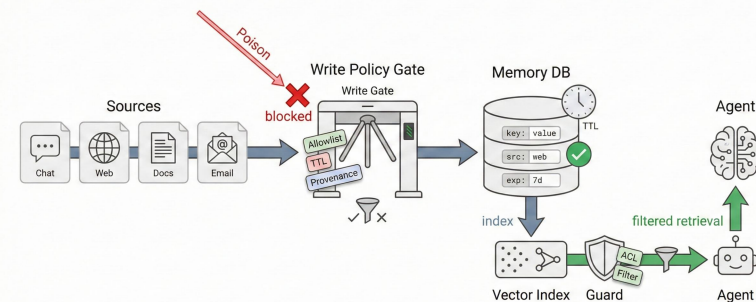


两类风险

- Memory Poisoning: 把恶意内容写入长期记忆/知识库，形成跨会话后门
- Embedding Weakness: 利用向量检索的相似度机制与 ACL/索引缺陷，让“错误/越权内容”更容易被召回并影响决策

典型防护

- 记忆写入策略：只允许写被批准的字段与格式（Allowlist 字段、TTL、来源标注）
- 记忆检索策略：检索出来的内容必须“可解释、可授权、可过滤”
- 关键记忆“签名/审批后写入：适用于一旦错了就会直接造成损失的字段



Unbounded Consumption: 资源耗尽 / Denial-of-Wallet / 无限循环



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

重点

Agent 最大的“工程型安全事故”之一不是泄密，而是“越跑越贵、越跑越久、停不下来”；所以必须给它加三道闸：**预算**、**循环上限**、**熔断降级**

先把“上限”写死

- 给每个任务/会话设硬上限：token、时间、钱、工具次数
- 跑可以，但跑到一定程度必须停（否则永远会遇到“无底洞任务”）

防止“停不下来”

- 设定：最大循环次数、最大重试次数
- 失败策略：指数退避（越失败等越久）+ 到点直接中止
- 不允许无限“再试一次”

发现异常就自动降级

- 触发条件
 - 调用速率异常飙升
 - 参数异常（比如特别大的top_k/特别贵的检索模式/异常目的地）
 - 错误率/超时率持续升高
- 动作：把模式从“自动 → 需确认 → 只读”逐级降级

1) Budget (预算配额)

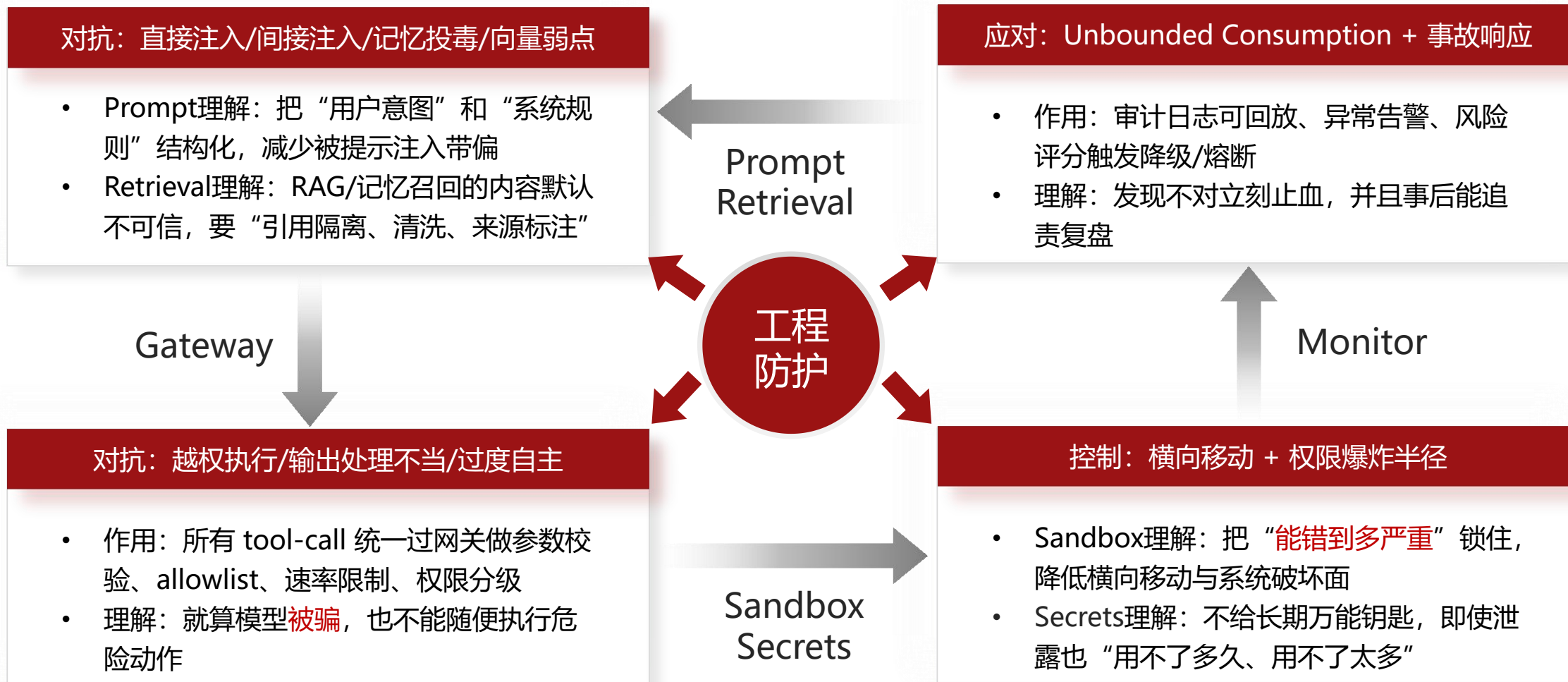
2) Loop Limits (循环限制)

3) Circuit Breaker (熔断器)

应对两种常见形态

- 成本 DoS（把系统拖死）：输入特别长、推理特别久、工具调用特别多
- Denial-of-Wallet：高频调用付费 API、付费检索、反复重试/反复搜索“直到找到答案”

工程防护栈 (Swiss Cheese / Defense-in-Depth)



不要指望某一个“万能防护”挡住所有风险：把防护做成**六层纵深**，每层都能拦下一部分攻击，叠加后才接近**可上线的安全性**



自主权边界与治理

自主性分级 + HITL 的工程难点：批准疲劳



批准疲劳

为什么“每步都问”反而更危险？

- 弹窗太多时，人会形成条件反射：“先点同意再说”，确认变成走过场

改法

- 风险聚合审批：思路--不要让用户对“每个小动作”确认，而是把一串动作合并成一个“证据包”来批
- 预览 + 证据：每次审批必须展示三样东西——将要执行的 API/tool-call+影响范围+来源引用
- Diff (差异/变更视图)：把 Agent “准备执行的写操作”用对比形式展示出来

治理机制：边界、预算、可追责、可停止

治理机制

解决什么问题：Agent 会持续运行、会调用工具，不能只靠模型“自觉”

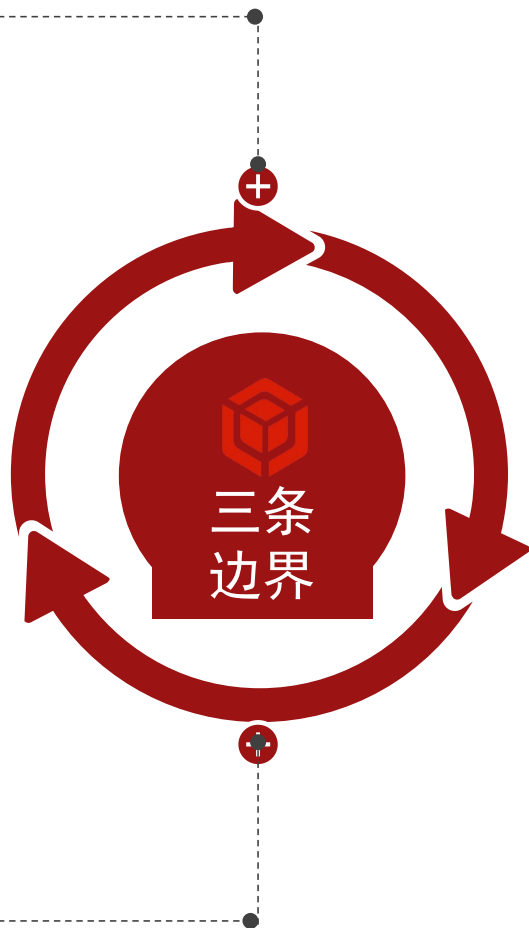
Scope（任务边界）

- 允许做哪些事、在什么范围内做
- 允许消耗多少成本
- 允许用哪些工具、哪些权限级别

Budget（资源/钱）

- 例子：只允许操作“某个 repo + 某个环境 + 某类工单”，不允许触达其他系统
- 例子：单任务最多检索 20 次、最多运行 5 分钟、最多花 \$2
- 例子：默认只读；写操作需确认；资金/不可逆需双人复核

Capability（权限边界）



关键治理组件

1

Kill Switch（紧急停止）

- 是什么：一键停机/暂停 Agent（停止新任务 + 终止当前任务 + 撤销凭证）
- 为什么必须有：当你发现异常（注入、循环、越权）时，先止血，否则自动化会放大事故
- 工程要点：支持按用户/租户/任务/工具维度“局部熔断”

2

审计日志（可回放的 tool-call 轨迹）

- 是什么：把每次工具调用记录成“可播放的时间线”：谁触发、调用了什么、参数是什么、返回了什么、造成了什么影响
- 为什么关键：不然你只能看最终结果，不知道中间哪一步被注入或越权

3

责任链（Delegation chain）

- 是什么：清楚记录谁委托（发起任务）→ 谁批准（放权限/放预算）→ 谁负责（Owner/On-call）
- 为什么必须：Agent 事故里最常见的问题是“责任扩散”（下一页多智能体会更严重）

多智能体治理：责任扩散与策略冲突



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

重点

多 Agent 必须 “统一调度 + 统一配额 + 可追责链路”，否则就是分布式失控

- 单 Agent 出事：基本能定位 “哪个任务、谁批准、哪个 tool-call”
- 多 Agent 出事：常见情况是 A 给 B 下任务、B 又调用 C，最后事故发生在 C 的动作上
- 结果：大家都 “参与了”，但很难一句话回答：到底谁是触发者、谁是批准者、谁该负责

新增风险一
责任扩散（谁造成的？）

- 不同 Agent 可能被不同目标驱动：一个追求 “快修复”，另一个强调 “合规/谨慎”，还有一个只追求 “成本最低”
- 典型冲突：
监控 Agent 频繁重启服务以恢复可用性；安全 Agent 认为异常重启可疑而触发封禁；结果互相对抗，系统抖动更严重
- 本质：没有统一的 “全局策略/优先级” 时，局部最优会互相破坏。

新增风险二
策略冲突
(Agent 间目标不一致)

- 多 Agent 最危险的 “工程事故” 之一：
A 发现问题 → 交给 B 排查 → B 让 C 去验证 → C 又把结果回给 A → A 再触发新一轮.....
- 结果：形成 闭环，不断调用工具/检索/执行，出现成本爆炸或服务被打满（也是 Unbounded Consumption 的多体版本）

新增风险三
涌现行为（互相调用形成环）

委托链规则（Delegation Policy：谁能委托谁）

- 做法：定义清晰的角色关系与上限
- 记录责任链：每个任务携带 root_request_id（最初委托）+ approver（批准者）+ executor（执行者）
- 价值：事故时能回答三句话：谁发起、谁批准、谁执行，避免责任扩散

跨 Agent 共享预算与熔断

- 共享预算：把 token/时间/钱/工具次数做成 “全局配额池”，多个 Agent 共用同一额度
- 共享熔断：一旦检测到异常（调用速率飙升、回路过长、错误率高），对整个 Agent 组降级/暂停，而不是只停一个
- 价值：防止 “两个 Agent 互相把对方当触发器”，把成本打爆

多 Agent 会更危险——因为它们会 “互相影响、互相调用、共同消耗资源”

治理抓手



评测、红队与运行时保障

Agent 安全评测：从“注入”到“行为安全”



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

这三类评测其实在回答三个不同问题

注入对抗

(AgentDojo——抗干扰题)

- 测点：当工具返回“不可信数据”（里面夹了恶意指令）时，Agent 能不能不被劫持、仍完成原任务
- 直观理解：你在正常办事，但别人塞给你一张“夹带私货的小纸条”，你会不会照做

外部工具/网页返回了“坏内容”，Agent 会不会被带偏去做坏事？

综合交互安全

(实操题)

- 测点：让 Agent 在很多交互环境里执行任务，覆盖多类安全风险，评估“整体行为安全分”。文中描述包含 349 个交互环境、2000 个测试用例、8 类风险。
- 直观理解：不是只防注入，而是看你在真实场景里会不会泄露数据/越权/做危险动作/被诱导做坏事

在各种真实任务环境里跑起来，Agent 的整体行为是否安全（泄露、越权、危险动作等）？

风险判断

(阅卷题)

- 测点：不给你“去执行”，而是给一段多轮交互记录，让模型判断这里有没有安全风险、属于哪类风险。数据集描述为 569 条多轮记录、27 个风险场景等。
- 直观理解：像安全审计员/风控模型：看日志就能识别风险（适合监控与审计）

给你一段“交互日志”，模型/系统能不能判断这段行为有没有安全风险、风险点在哪？

总结

三种评测对应三件事：防注入、看行为、会审计

Runtime Safety: 监控、降级与事故响应



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

01

在线监控：三类异常

- 异常频率：突然调用次数飙升
- 异常参数：参数看起来“太大/太危险/太不合理”
- 异常目的地：请求跑向不该去的地方

02

在线监控：风险评分

- 核心思路：不要“要么全自动要么全停”，而是分档降级，先止血
- 低风险：继续自动（但记录/告警）
- 中风险：切到“执行前确认”（HITL）
- 高风险：切到“只读模式”（禁止写/钱/不可逆）或直接暂停任务
- 极端：触发熔断 / Kill Switch

事故响应
SOP

把安全运营做成“固定动作”，
避免出事时手忙脚乱

- 发现：来源为告警（异常频率/参数/目的地）、用户反馈、监控仪表盘风险分飙升
- 止血：第一目标，立刻把损失停住
- 溯源：看可回放的 tool-call 轨迹
- 修复：修复规则/策略/网关校验、检索过滤、权限 scope、记忆清理、prompt 模板、模型/工具 bug
- 复盘：根因 + 影响面 + 为什么没提前挡住 + 新增防线/测试用例

上线后的安全靠“盯 tool-call + 快速降级 + 标准化止血流程”：发现异常先降权限/限流，保住系统与钱包，然后再溯源修复



未来挑战与总结

未来研究方向

核心

未来的 Agent 安全要从“靠规则拦”走向“可验证、可治理、可规模化监督”：既要防注入，也要让每一步行动可证明、可审计、可控成本

注入与可验证行动

- 既要不被外部内容带偏，也要让每一步动作“拿得出证据”
- 要点包括：
 - 内容≠指令：引用/证据与操作命令强隔离
 - 自动红队/对抗训练：持续更新注入样本
 - Tool-call 收据：输入/输出/权限/影响范围可追溯（可审计）

可执行治理策略与最小暴露隐私

- 把合规/权限写成“能运行的规则”，并让敏感数据在受控边界内完成推理
- 要点包括
 - Policy-as-Code：可版本化、可回滚、可审计
 - Learning 优化：在安全约束下减少打扰（降低批准疲劳）
 - 最小暴露检索：只返回必要片段/统计，隔离环境处理机密

多智能体协作治理与可扩展监督

- 多体系统要“统一调度+共享约束”，监督要“少人力覆盖长链条”
- 要点包括
 - Coordinator：统一策略/预算/熔断
 - 冲突仲裁+环路检测：防互相递归失控
 - 分层抽检：证据包审批 + 风险评分 + 事后回放审计

鲁棒防注入
(尤其间接注入)

可验证工具调用
(动作可证明/证据可核查)

Policy-as-Code +
Learning

可扩展监督
(低成本审查长链条动作)

多智能体安全
(冲突解决、责任分配、涌现控制)

机密执行/隐私
计算

01

Agent 的风险本质：从“说错话”升级为“做错事 + 记错事 + 持续做错事”，核心来自行动能力、长期记忆与外部输入面

02

工程落地要点：用纵深防御把风险关进笼子——最小权限与分级授权、注入隔离与记忆治理、预算/熔断与沙箱、全链路审计

03

上线保障闭环：以治理与合规为边界、以评测红队与在线监控为抓手，做到可降级、可追责、可持续改进